Сегментация объектов в видео

SAM 2 и его современные улучшения

Александр Савельев

Курс: Обработка видео в контексте машинного обучения и ИИ
Лекция 6

План лекции

Главный вопрос: *как, зная объект в одном кадре, получить его точную маску во всём видео — через окклюзии, двойников и смену сцены*.

1. Постановки: VOS, интерактивная сегментация, по тексту, VIS, концепты
2. SAM 2 подробно: из чего состоит, что лежит в памяти, как обрабатывается один кадр
3. Где SAM 2 ломается и как его чинят без переобучения: SAMURAI, SAM2Long, DAM4SAM, SeC
4. SAM 3 и SAM 3.1 — коротко, что вышло (подробно — в конце, если останется время)
5. Сегментация по тексту и VIS — коротко
6. Метрики $\mathcal{J}$, $\mathcal{F}$, $\mathcal{J}\&\mathcal{F}$; датасеты; состояние области в 2026

1. Что значит «сегментировать объект в видео»



Masklet — маска объекта во времени

Видео — кадры $I_1, \ldots, I_T$, кадр $I_t \in \mathbb{R}^{H \times W \times 3}$ (высота H, ширина W, 3 канала цвета). Masklet объекта — последовательность масок $M_1, \ldots, M_T$, где $M_t \in \{0,1\}^{H \times W}$: $M_t(\mathbf{x}) = 1$, если пиксель $\mathbf{x}$ кадра t принадлежит объекту. Если объект закрыт или ушёл из кадра, M_t — пустая маска (все нули).

Отличие от лекции 5: там объект — бокс, здесь — *точный контур* каждого пикселя. Номер объекта остаётся тем же, что и в трекинге.

1. Пять постановок: что дано и что найти



- **Semi-supervised VOS** — классика: маска M_1 объекта в первом кадре дана, найти $M_2, \ldots, M_T$.
- **Интерактивная (PVS — promptable video segmentation)**: пользователь кликает в любом кадре и исправляет ошибки. Это постановка SAM 2.
- **Referring VOS**: объект задан фразой; **VIS**: все объекты из списка классов; **PCS** (promptable concept segmentation, SAM 3): все экземпляры концепта по короткой фразе.

1. Формально: semi-supervised VOS

Задача

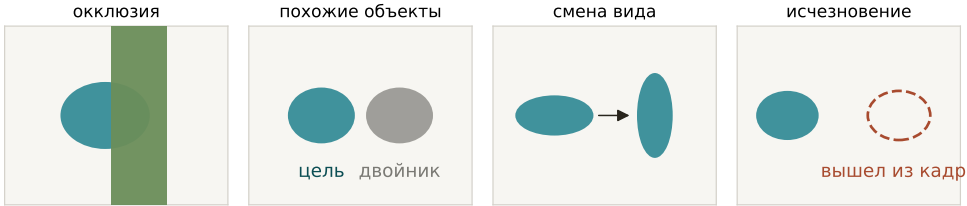
Дано: кадры $I_1, \ldots, I_T$ и маска первого кадра $M_1 \in \{0, \ldots, K\}^{H \times W}$ — в каждом пикселе номер объекта (0 — фон, $1, \ldots, K$ — объекты).
Найти: маски $\hat{M}_2, \ldots, \hat{M}_T$ того же вида: каждому пикселю каждого кадра — номер объекта или фон.

Онлайн-режим

Маска $\hat{M}_t$ строится только по кадрам $I_1, \ldots, I_t$ и уже готовым маскам $\hat{M}_1, \ldots, \hat{M}_{t-1}$. Будущее не видим — так работает видеопоток и все методы этой лекции.

Ключевая трудность: истинная маска есть *только в кадре 1*. Всё остальное модель строит из собственных предсказаний — и ошибки накапливаются.

1. Зачем это нужно и что мешает



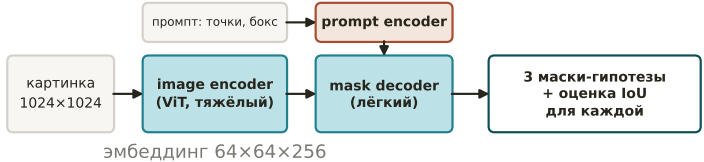
Где применяется

- монтаж и ротоскопинг: вырезать человека из фона
- удаление объекта: маска + inpainting (ProPainter)
- разметка данных: клик вместо контура на каждом кадре
- медицина, спорт, робототехника

Что ломает методы

- объект закрыт и появляется снова
- рядом похожий объект (двойник)
- объект меняет вид, повернулся
- смена сцены, длинные видео

2. Отправная точка: SAM для одной картинки



- **Image encoder** — большой ViT, считается *один раз* на картинку.
- **Prompt encoder**: клик (координата + «объект / фон») и бокс → векторы-токены.
- **Mask decoder** — лёгкий трансформер: токены промпта ↔ признаки. Новый клик — миллисекунды.
- Клик неоднозначен (пуговица? рубашка? человек?) ⇒ **три маски-гипотезы** и предсказанный IoU каждой.

SAM ничего не знает о времени. SAM 2 добавляет ровно то, чего не хватает: память о прошлых кадрах.

2. SAM 2 целиком Ravi et al., 2024

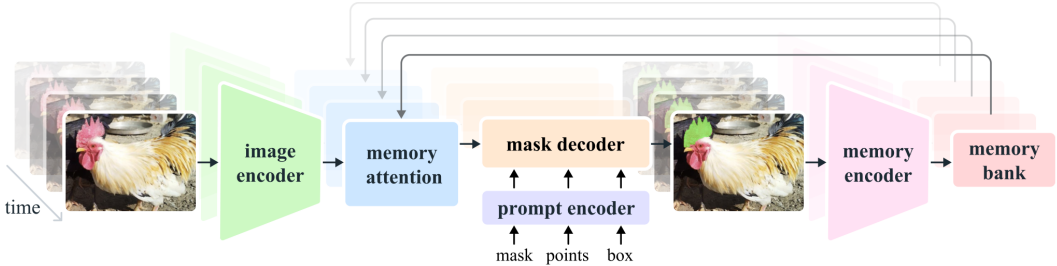
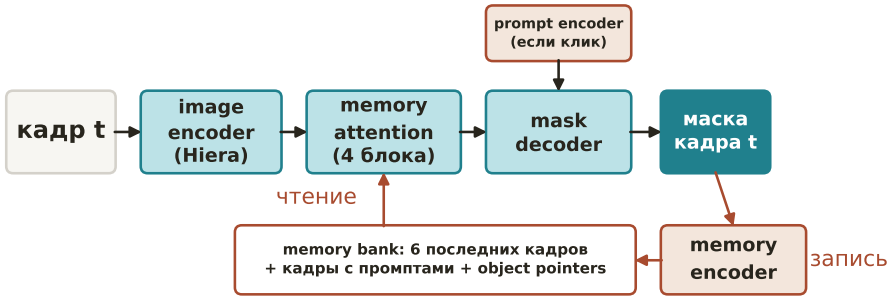


Рисунок: Ravi et al. SAM 2, arXiv 2408.00714

Кадры идут потоком, один за другим. От SAM остались image encoder, prompt encoder и mask decoder. Новое — три блока памяти: **memory attention** (кадр читает память), **memory encoder** (кадр с маской записывается) и **memory bank** (где лежат записи). Дальше разберём каждый блок.

2. Поток данных на одном кадре



- **Чтение** (до декодера): признаки кадра *t* смотрят в память — «где здесь объект, похожий на запомненный?»
- **Запись** (после декодера): маска кадра *t* кодируется и кладётся в память для следующих кадров.
- Клик не обязателен: без кликов модель сама протаскивает объект по видео.

2. Image encoder: Hiera и его предобучение

Что делает

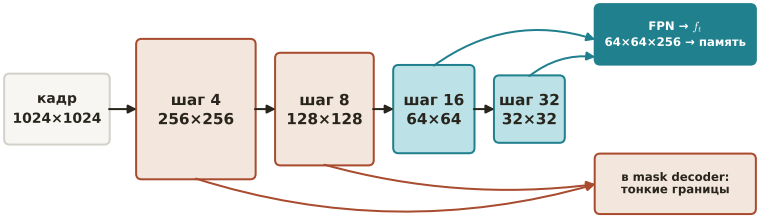
$\mathbf{F}_t = \mathrm{Enc}(I_t)$: кадр $1024 \times 1024 \to$ признаки четырёх масштабов, с шагом 4, 8, 16, 32 пикселя. Hiera — иерархический ViT: как пирамида у CNN, разрешение падает от стадии к стадии.

Предобучение MAE (masked autoencoder) — без разметки

Картинку режут на квадраты 32×32 пикселя и 60% из них удаляют. Сеть видит только оставшиеся и должна восстановить пиксели удалённых. Чтобы дорисовать закрытое, ей приходится понимать формы и контекст. После обучения декодер MAE выбрасывают, энкодер берут в SAM 2.

Зачем: размеченных масок мало, а картинок без разметки — миллионы. MAE даёт энкодеру «зрение» до того, как его начнут учить сегментации.

2. Image encoder: что получается на выходе



- Стадии с шагом 16 и 32 сливаются (FPN) в главную карту $\mathbf{f}_t \in \mathbb{R}^{64\times64\times256}$: $1024/16 = 64$ клетки по стороне, 256 каналов. Её читает память.
- Карты с шагом 4 и 8 идут прямо в mask decoder — для тонких границ маски.
- Энкодер не знает про объект и промпты: $\mathbf{F}_t$ считается один раз и общий для всех объектов в кадре.

Это самая дорогая часть. Размер Hiera задаёт версию модели: tiny 38.9M параметров и 91 кадр/с, large 224.4M и 39.5 кадра/с (SAM 2.1, A100).

2. Что приходит в mask decoder

Вход	Откуда	Зачем
$\mathbf{X}$: $64 \times 64 \times 256$	энкодер $\rightarrow$ memory attention (читает memory bank)	«где здесь объект»
$\mathbf{F}^{(8)}$: $128 \times 128 \times 64$, $\mathbf{F}^{(4)}$: $256 \times 256 \times 32$	энкодер напрямую, в обход памяти	тонкие границы
токены промпта	prompt encoder: клики, бокс	правка пользователя
6 выходных токенов	обучаемые веса декодера	«бланки» для ответов

Memory bank *не* подаётся в декодер напрямую: всё, что известно о прошлых кадрах, уже «вписано» в $\mathbf{X}$ на шаге memory attention (разберём после декодера). Энкодер даёт декодеру смысл — через $\mathbf{X}$ — и детали границы — через $\mathbf{F}^{(4)}, \mathbf{F}^{(8)}$.

Кадр 1: памяти ещё нет, $\mathbf{X} = \mathbf{f}_t$+обучаемый вектор «нет памяти».

2. Токены промпта и выходные токены

Токены промпта (prompt encoder)

Клик в точке (x, y): $\mathbf{p} = \mathrm{PE}(x, y) + \mathbf{l}_{\text{метка}}$, где PE — позиционный код координаты, $\mathbf{l}$ — обучаемый вектор «объект» или «фон». Бокс — два токена (углы). В конец добавляется токен-заглушка «нет точки» (все случаи — на следующем слайде). Маска-подсказка (если есть) сжимается свёртками и прибавляется к $\mathbf{X}$; иначе прибавляется обучаемый вектор «нет маски».

Выходные токены — обучаемые «бланки»

Шесть векторов по 256 чисел, одинаковых для любого кадра: токен объекта, IoU-токен и 4 токена масок (1 для одиночной маски + 3 для гипотез; при трекинге SAM 2.1 берёт тройку). Сами по себе они ничего не знают; в декодере они собирают информацию о кадре, и каждый отвечает на свой вопрос.

Идея та же, что у запросов DETR и Mask2Former: обучаемый вектор-«вопрос» превращается в ответ после attention к признакам.

2. Токены промпта: три случая



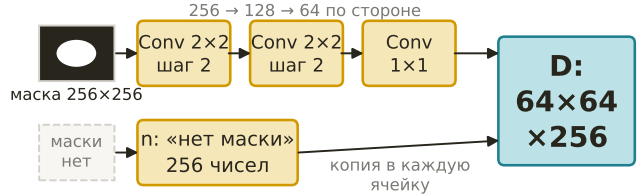
PE — где кликнули (формула, не обучается). **I** — что означает клик: 4 обучаемых вектора (объект, фон, два угла бокса). **n** — обучаемая заглушка. Каждый токен — 256 чисел.

2. Позиционный код и плотная подсказка

Позиционный код точки (не обучается)

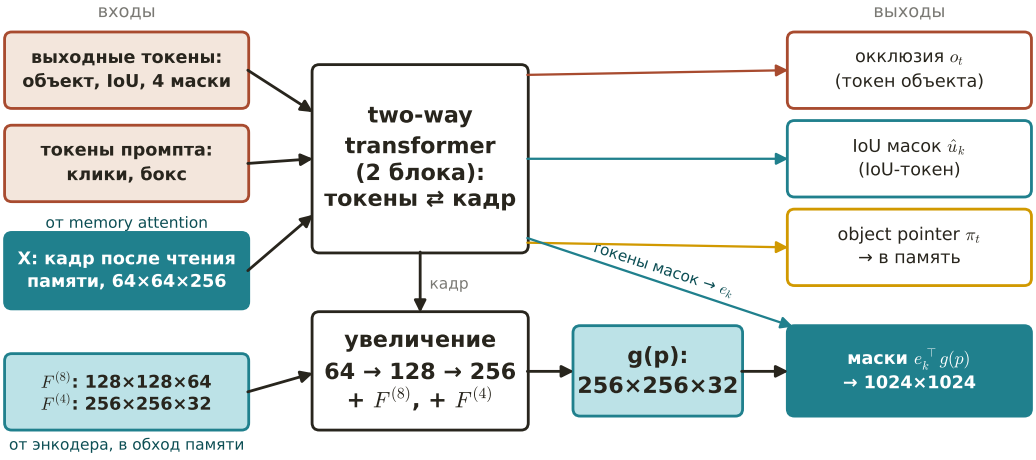
$$\mathrm{PE}(x,y) = \big[\sin(2\pi\,\mathbf{u}B),\ \cos(2\pi\,\mathbf{u}B)\big] \in \mathbb{R}^{256}, \qquad \mathbf{u} = \Big(2\tfrac{x}{W} - 1,\ 2\tfrac{y}{H} - 1\Big)$$

u — координата клика в $[-1, 1]$; $W = H = 1024$ — размер входа; $B \in \mathbb{R}^{2\times 128}$ — случайная матрица, выбрана один раз и не обучается; sin и cos дают $128 + 128 = 256$ чисел. Тот же код считается для каждой ячейки карты.



Плотная подсказка: $S = \mathbf{X} + \mathbf{D}$. Свёртки и **n** «нет маски» обучаемы (золотые).

2. Mask decoder изнутри



Токены и карта **X** обмениваются информацией; затем карта растёт $64 \rightarrow 256$ с картами энкодера — так возвращаются детали границы.

2. Two-way transformer и увеличение разрешения

Один блок (их 2); T — токены, $S = $ X+плотная подсказка

$$
\begin{aligned}
&1)\ T \leftarrow T + \mathrm{SelfAttn}(T) && \text{токены советуются между собой}\\
&2)\ T \leftarrow T + \mathrm{CrossAttn}(T, S) && \text{токены смотрят в кадр}\\
&3)\ T \leftarrow T + \mathrm{MLP}(T)\\
&4)\ S \leftarrow S + \mathrm{CrossAttn}(S, T) && \text{кадр смотрит в токены}
\end{aligned}
$$

$T = [\mathbf{t}_{\text{obj}}, \mathbf{t}_{\text{IoU}}, \mathbf{t}_1, \ldots, \mathbf{t}_4, \mathbf{p}_1, \ldots]$: выходные токены и токены промпта. В шаге 4 ячейки кадра узнают, что ищем.

Увеличение и головы

$$S_{64} \xrightarrow{\uparrow 2} +\mathbf{F}^{(8)} \xrightarrow{\uparrow 2} +\mathbf{F}^{(4)} = \mathbf{g} \in \mathbb{R}^{256\times256\times32}$$

$\uparrow 2$ — транспонированная свёртка, удваивает сторону и уменьшает число каналов. Головы: $\mathbf{e}_k = \mathrm{MLP}_k(\mathbf{t}_k) \in \mathbb{R}^{32}$; $\hat{\mathbf{u}} = \mathrm{MLP}(\mathbf{t}_{\text{IoU}})$; $o_t = \mathrm{MLP}(\mathbf{t}_{\text{obj}})$.

2. Зачем нужен каждый токен (1/2)

Токен	Что выходит	Зачем нужен
$\mathbf{t}_{\text{obj}}$	логит $o_t \in \mathbb{R}$	виден ли объект. Если $o_t \leq 0$: маска пустая, pointer — вектор «нет объекта»
$\mathbf{t}_{\text{IoU}}$	$\hat{\mathbf{u}} \in \mathbb{R}^4$: по числу на маску	качество каждой маски без истины; по нему выбираем гипотезу
$\mathbf{t}_1, \ldots, \mathbf{t}_4$	$\mathbf{e}_k \to$ маски	сами маски (следующий слайд)
$\mathbf{p}_1, \mathbf{p}_2, \ldots$	не используется	несут клики: через attention сообщают токенам и кадру, что ищем

Все токены проходят один two-way transformer; различаются только головой на выходе и слагаемым функции потерь (слайд «Как SAM 2 учили»).

2. Зачем нужен каждый токен (2/2): токены масок

Одна маска или три гипотезы

- $\mathbf{t}_1$ — **одна маска**, когда промпт однозначен: 2 клика и больше или бокс (бокс = 2 точки).
- $\mathbf{t}_2, \mathbf{t}_3, \mathbf{t}_4$ — **три гипотезы**, когда кликов 0 или 1, в том числе при трекинге без кликов. Дальше нумеруем их $k = 1, 2, 3$.

Пример: один клик по рубашке

Гипотезы: рубашка, человек целиком, группа людей; IoU-токен оценивает их как $\hat{u} = (0.62,\ 0.91,\ 0.35) \Rightarrow$ берём «человека».

Выбранный токен маски работает дважды: даёт маску кадра и после проекции становится object pointer $\boldsymbol{\pi}_t$ для памяти.

2. Как из токена получается маска

Маска — скалярное произведение токена и пикселя

$$\hat{y}_k(\mathbf{p}) = \mathbf{e}_k^\top \mathbf{g}(\mathbf{p}), \qquad \hat{M}_k(\mathbf{p}) = \big[\sigma(\hat{y}_k(\mathbf{p})) > 0.5\big], \qquad k = 1, 2, 3$$

$\mathbf{e}_k \in \mathbb{R}^{32}$ — токен k-й маски после MLP; $\mathbf{g}(\mathbf{p}) \in \mathbb{R}^{32}$ — признак ячейки $\mathbf{p}$ карты 256×256 с прошлого слайда; $\sigma(z) = 1/(1 + e^{-z})$ — сигмоида; $[\cdot]$ — 1, если условие верно. Маска 256×256 затем билинейно увеличивается до 1024×1024.

Выбор одной из трёх

$$k^* = \arg \max_{k \in \{1,2,3\}} \hat{u}_k$$

$\hat{u}_k$ — IoU маски k с истиной, *предсказанный* IoU-головой (истины на тесте нет). Пример: $\hat{u} = (0.95, 0.71, 0.40) \Rightarrow k^* = 1$.

Пиксель попадает в маску, если его признак «смотрит в ту же сторону», что и токен маски. Та же идея, что у масок Mask2Former (раздел 5).

2. Голова окклюзии и object pointer

Occlusion head: виден ли объект

$o_t = \mathrm{MLP}(\mathbf{e}_{\mathrm{obj}}) \in \mathbb{R}$ — логит по токену объекта. Если $o_t < 0$, объект считается невидимым: маска кадра — пустая, даже если какие-то пиксели похожи.

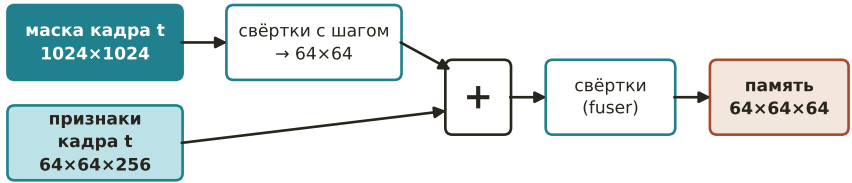
Зачем: в SA-V 42.5% masklets хотя бы раз исчезают. Без такой головы модель «прилипала» бы к ближайшему похожему объекту.

Object pointer: краткое описание объекта

$\boldsymbol{\pi}_t = \mathrm{Proj}(\mathbf{e}_{k^*}) \in \mathbb{R}^{256}$ — выходной токен выбранной маски. В памяти он режется на 4 токена по 64 числа — под размер памяти.

Карты памяти отвечают на вопрос «*где* был объект», pointer — «*что* это за объект». Pointer не привязан к месту в кадре, поэтому помогает найти объект после того, как тот переместился.

2. Memory encoder: что записываем о кадре



Запись кадра в память

$$\mathbf{m}_t = \mathrm{Fuse}\big(\mathbf{f}_t + \mathrm{Down}(\hat{M}_t)\big) \in \mathbb{R}^{64\times64\times64}$$

$\hat{M}_t$ — выбранная маска кадра; Down — свёртки с шагом, сжимающие маску до 64×64 и 256 каналов; $\mathbf{f}_t$ — признаки кадра от энкодера; Fuse — ещё несколько свёрток и проекция в 64 канала.

Память хранит *признаки кадра, раскрашенные маской*: «здесь ячейки такого вида, и они были объектом». 64 канала вместо 256 — память в 4 раза компактнее.

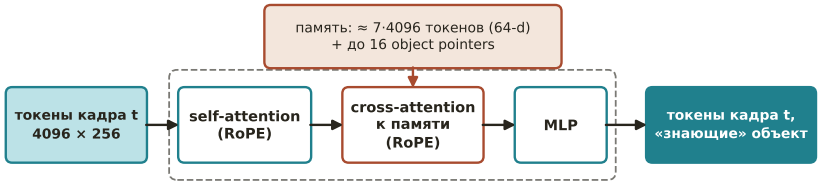
2. Memory bank: какие кадры помнит SAM 2



- **FIFO из $N = 6$ последних кадров**: новый вытесняет самый старый.
- **Кадры с промптами** (до M штук): кадр 1 и кадры, где пользователь кликал. Не вытесняются — это «истина».
- **Object pointers** последних кадров (до 16).
- Каждому кадру памяти добавляется *временное кодирование*: насколько кадров назад он был.

Всего около 7 кадров по $64 \cdot 64 = 4096$ элементов. Объём памяти не растёт с длиной видео — цена: кадры 2–33 на рисунке забыты, даже если были очень полезны. Это главная мишень улучшений в разделе 3.

2. Memory attention: чтение памяти



× 4 блока; после каждого слагаемого --- остаточная связь и нормализация

Один блок (повторяется $L = 4$ раза)

$\mathbf{X}^{(0)}$ — признаки $\mathbf{f}_t$, развёрнутые в $64 \cdot 64 = 4096$ токенов по 256 чисел.

$$\mathbf{X} \leftarrow \mathbf{X} + \mathrm{SelfAttn}(\mathbf{X}), \quad \mathbf{X} \leftarrow \mathbf{X} + \mathrm{CrossAttn}(\mathbf{X}, \mathcal{B}_t), \quad \mathbf{X} \leftarrow \mathbf{X} + \mathrm{MLP}(\mathbf{X})$$

$\mathcal{B}_t$ — все токены памяти: карты **m** кадров памяти и object pointers.

2. Cross-attention: как кадр читает память

Формула

$$\mathrm{CrossAttn}(\mathbf{X}, \mathcal{B}) = \mathrm{softmax}\Big(\frac{(\mathbf{X}W_Q)(\mathcal{B}W_K)^\top}{\sqrt{d}}\Big)\,\mathcal{B}W_V$$

$\mathbf{X}$ — токены кадра (по строке на токен, 4096 строк); $\mathcal{B}$ — токены памяти; W_Q, W_K, W_V — обучаемые матрицы; d — длина ключа; softmax — по токенам памяти (по строке).

Идея: каждый токен кадра ищет в памяти похожие на него токены и забирает их содержимое. Если похожие токены памяти были объектом, «объект» попадёт и в него. Так маска кадра 1 «доезжает» до кадра t без переноса по соседним кадрам.

Деление на $\sqrt{d}$: скалярное произведение d случайных чисел растёт как $\sqrt{d}$; без деления softmax отдавал бы почти весь вес одному токену.

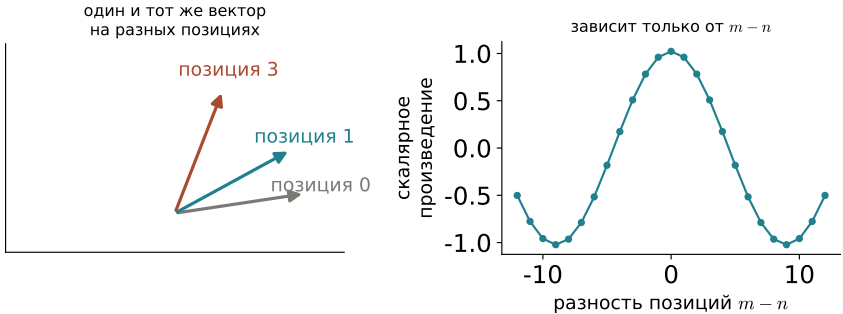
2. Cross-attention по шагам

Шаг	Что происходит для токена кадра j
запрос	$\mathbf{q}_j = \mathbf{x}_j W_Q$: «что я ищу»
ключ, значение	$\mathbf{k}_i = \mathbf{b}_i W_K$ («как выглядит»), $\mathbf{v}_i = \mathbf{b}_i W_V$ («что известно») для каждого токена памяти i; 64 $\rightarrow$ 256 чисел
сходство	$S_{ij} = \mathbf{q}_j^\top \mathbf{k}_i / \sqrt{d}$ (+ RoPE, следующий слайд)
веса	$W_{ij} = \exp(S_{ij}) / \sum_n \exp(S_{nj})$: доли, в сумме 1
чтение	$\mathbf{r}_j = \sum_i W_{ij} \mathbf{v}_i$, затем $\mathbf{x}_j \leftarrow \mathbf{x}_j + \mathbf{r}_j$

Пример: 4 токена памяти

Сходства $(-0.2, -0.5, -2.0, -3.0) \rightarrow$ веса $(0.51, 0.38, 0.08, 0.03)$. Токены 1, 2 были объектом (метка 1), 3, 4 — фоном (0): прочитано $0.51 + 0.38 = 0.89$ — почти наверняка объект.

2. RoPE: зачем позиция в attention



Attention сам не знает, где в кадре стоит токен: переставьте токены — ответ не изменится. RoPE (rotary position embedding) поворачивает вектор на угол, пропорциональный позиции (слева). Тогда скалярное произведение двух векторов зависит только от того, насколько далеко они друг от друга (справа).

2. RoPE: формула

Поворот пар координат

Координаты вектора разбиты на пары; пара (a, b) токена на позиции m поворачивается на угол $m\theta$:

$$R_m \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix}, \qquad (R_m \mathbf{q})^\top (R_n \mathbf{k}) = \mathbf{q}^\top R_{n-m}\, \mathbf{k}$$

θ — своя частота для каждой пары; $R_m^\top R_n = R_{n-m}$, потому что повороты складываются: «назад на m», затем «вперёд на n».

«Ключ на 3 клетки левее» значит одно и то же в любом месте кадра. В SAM 2 RoPE двумерный (половина пар — по x, половина — по y); object pointers его не получают — они «без адреса».

2. Сколько это стоит

Подсчёт для одного кадра и одного объекта

Запросов: $64 \cdot 64 = 4096$. Ключей памяти: ≈ 7 кадров $\times$ 4096 $+$ до 16 pointers $\times$ 4 токена $\approx 28\,700$. Матрица сходств cross-attention: $4096 \times 28\,700 \approx 1.2 \cdot 10^8$ чисел на блок, 4 блока.

- Стоимость *не зависит* от длины видео: память фиксированного размера.
- Энкодер общий для всех объектов, а memory attention, decoder и memory encoder — *отдельно на каждый объект*: 10 объектов $\approx$ 10 проходов этой части.
- Скорость SAM 2 (A100): Hiera-B+ 43.8 кадра/с, Hiera-L 30.2 кадра/с.

Линейный рост по числу объектов — проблема для сцен с толпой. Её решает Object Multiplex в SAM 3.1 (раздел 4 и дополнительная часть).

2. Один кадр SAM 2 целиком

Алгоритм: обработка кадра t для одного объекта

вход: кадр I_t; memory bank $\mathcal{B}_t$; клики в кадре t (если есть)
выход: маска $\hat{M}_t$; обновлённая память

1. $\mathbf{F}_t \leftarrow \mathrm{Enc}(I_t)$ — один раз на кадр, общий для всех объектов.
2. $\mathbf{X} \leftarrow \mathrm{MemAttn}(\mathbf{f}_t, \mathcal{B}_t)$ — 4 блока self/cross/MLP.
3. Если есть клики: токены промпта $\leftarrow \mathrm{PromptEnc}(\text{клики})$.
4. $\{\hat{M}_k, \hat{u}_k\}_{k=1}^{3}, o_t, \pi_t \leftarrow \mathrm{Dec}(\mathbf{X}, \text{токены}, \mathbf{F}_t^{(4)}, \mathbf{F}_t^{(8)})$.
5. $k^* \leftarrow \arg\max_k \hat{u}_k$. Если $o_t < 0$: $\hat{M}_t \leftarrow \varnothing$, иначе $\hat{M}_t \leftarrow \hat{M}_{k^*}$.
6. $\mathbf{m}_t \leftarrow \mathrm{MemEnc}(\mathbf{f}_t, \hat{M}_t)$; положить $(\mathbf{m}_t, \pi_t)$ в FIFO; если были клики — в кадры с промптами.

$\mathbf{F}_t^{(4)}, \mathbf{F}_t^{(8)}$ — карты энкодера с шагом 4 и 8. Без кликов (шаг 3 пропущен) это и есть semi-supervised VOS.

2. Интерактивный режим: правка одним кликом

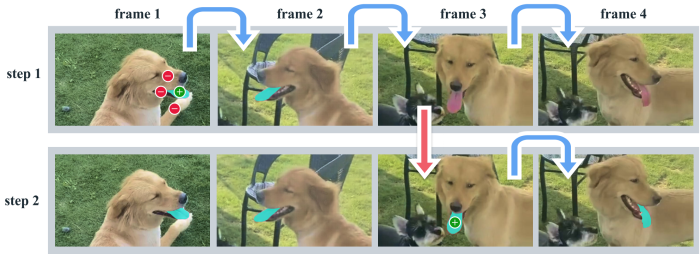


Рисунок: Ravi et al. SAM 2, arXiv 2408.00714

Шаг 1: клики в кадре 1, SAM 2 ведёт объект дальше. В кадре 3 маска «уехала» на другую собаку. Шаг 2: один клик в кадре 3 — кадр становится кадром с промптом, попадает в память навсегда, и SAM 2 переразмечает кадры дальше. Старые методы требовали бы перезапуска с нуля.

2. Как SAM 2 учили

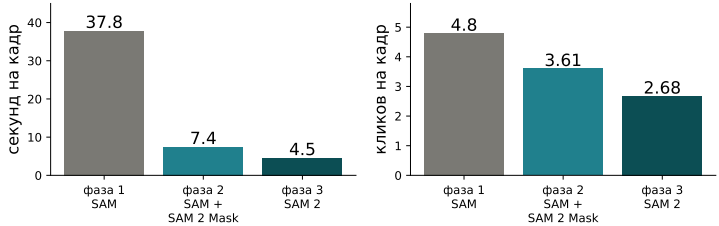
- **Картинки и видео вместе**: SA-1B (картинка = видео из одного кадра) и SA-V + открытые VOS-датасеты.
- **Последовательности по 8 кадров** (в SAM 2.1 — по 16): на 1–2 кадрах симулируются клики, бокс или маска, остальные кадры — «смотри сам».
- **Клики симулируются по ошибкам**: новый клик ставится в область, где предсказание расходится с истиной — как делает человек.

Функция потерь

$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{focal+dice}}(\hat{M}, M) + \lambda_2 |\hat{u} - \text{IoU}(\hat{M}, M)| + \lambda_3 \text{CE}(o, \mathbb{1}[\text{объект виден}])$
Маска — focal + dice (пиксели и перекрытие); IoU-голова учится предсказывать настоящий IoU своей маски; голова окклюзии — бинарная классификация. λ — веса слагаемых.

Лосс маски считается только для лучшей из трёх гипотез — чтобы гипотезы специализировались на разных трактовках клика.

2. Data engine: модель помогает размечать себя



- Фаза 1: SAM на каждом кадре отдельно. Фаза 2: SAM 2 Mask протаскивает маску по видео. Фаза 3: полный SAM 2 с кликами. Итог — в 8.4 раза быстрее.
- **SA-V**: 50.9K видео, 196 часов, 642.6K masklets (190.9K ручных + 451.7K автоматических), 35.5M масок, 47 стран, разметка с частотой 6 кадров/с.

Цикл «модель размечает → человек правит → модель дообучается» — главный источник прорыва. Ровно так же потом сделан SAM 3.

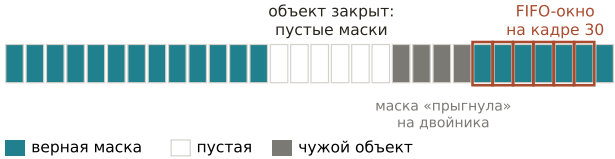
2. Результаты SAM 2 и SAM 2.1

Метод	DAVIS 17	MOSE	SA-V test	LVOS	кадров/с
до SAM 2: специализированные VOS-модели с памятью					
STCN (2021)	85.4	52.5	62.5	—	≈20
XMem (2022)	86.0	59.6	62.3	—	≈23
Cutie-base+ (2024)	88.1	71.7	62.8	—	—
SAM 2 Hiera-B+	90.2	76.6	77.0	78.0	43.8
SAM 2 Hiera-L	90.7	77.9	78.4	78.0	30.2
SAM 2.1 tiny / large	—	71.8 / 74.6	76.5 / 79.5	77.3 / 80.6	91.2 / 39.5

$\mathcal{J}\&\mathcal{F}$, %. Строки 1–5: таблица 6 SAM 2; SAM 2.1: README facebookresearch/sam2 (LVOS v2). Протоколы немного различаются.

SAM 2.1 (сентябрь 2024): те же блоки, но обучение на 16 кадрах и аугментации с похожими и мелкими объектами. Именно его веса берут все улучшения раздела 3.

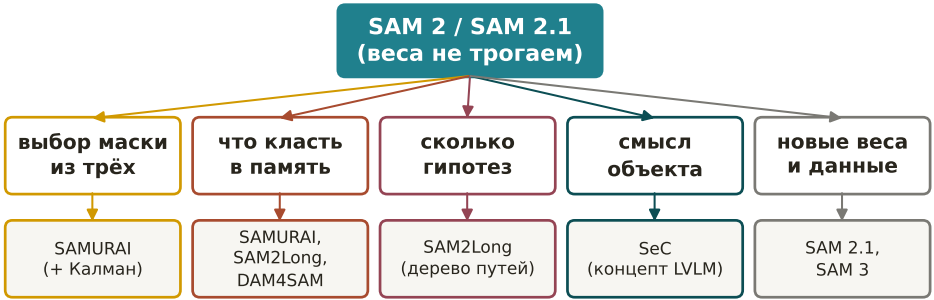
3. Где SAM 2 ломается



- **FIFO не смотрит на качество**: в память попадают и пустые кадры окклюзии, и кадры, где маска «прыгнула» на двойника. Дальше модель уверенно ведёт *не тот* объект.
- **Жадный выбор**: на каждом кадре берётся одна маска из трёх; ошибку нельзя отыграть назад.
- **Нет смысла объекта**: только «похожа ли ячейка». После смены сцены или ракурса похожих пикселей может не остаться.

На MOSEv2 (2025: окклюзии, толпы, смена планов) SAM 2 падает с 76.4 до 50.9 $\mathcal{J}\&\mathcal{F}$. Cutie — с 87.9 на DAVIS до 43.9.

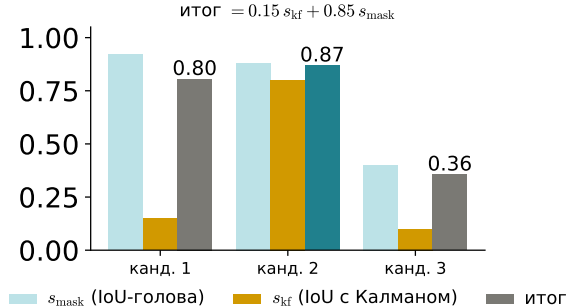
3. Карта улучшений: что именно меняют



Общий приём 2024–2026: **веса SAM 2.1 не трогаем** (training-free) и меняем только *правила*: какую маску выбрать, что записать в память, сколько гипотез держать. Это дёшево и работает поверх любой версии SAM 2 — а в 2026 и поверх трекера SAM 3.

3. SAMURAI: добавить движение Yang et al., 2024





Проблема: двойник выглядит так же, и IoU-голова SAM 2 даёт ему 0.92 против 0.88 у настоящего объекта. Но двойник стоит *не там, куда объект должен был сдвинуться*. Фильтр Калмана из лекции 5 знает траекторию бокса — добавим это знание в выбор маски.

3. SAMURAI: движение и выбор маски

1. Движение бокса — фильтр Калмана

Состояние $\mathbf{x} = [x, y, w, h, \dot{x}, \dot{y}, \dot{w}, \dot{h}]^\top$: центр (x, y), ширина w, высота h бокса и их скорости (точка сверху — изменение за кадр). Шаг predict даёт бокс $\hat{\mathbf{b}}_t$ — где объект должен оказаться в кадре t, если движется как раньше.

2. Выбор маски

$$s_k^{\text{kf}} = \text{IoU}\big(\text{box}(\hat{M}_k), \hat{\mathbf{b}}_t\big), \qquad k^* = \arg\max_k \big[\alpha\, s_k^{\text{kf}} + (1-\alpha)\, s_k^{\text{mask}}\big]$$

$\text{box}(\hat{M}_k)$ — бокс вокруг k-й маски; s_k^{kf} — согласие с движением; s_k^{mask} — IoU, предсказанный головой SAM 2; $\alpha = 0.15$ — вес движения.

3. SAMURAI: пример и отбор памяти

Пример с рисунка «SAMURAI: добавить движение»

$$\text{кандидат 2:} \quad 0.15 \cdot 0.80 + 0.85 \cdot 0.88 = 0.87$$

$$\text{двойник:} \quad 0.15 \cdot 0.15 + 0.85 \cdot 0.92 = 0.80$$

Голова SAM 2 чуть больше «верит» двойнику (0.92 против 0.88), но двойник далеко от прогноза Калмана (0.15) — побеждает настоящий объект.

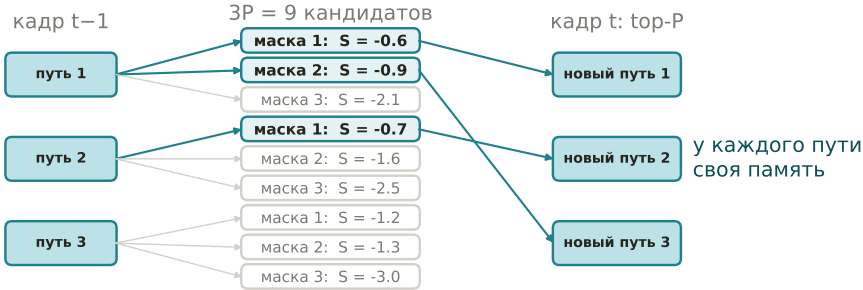
3. Отбор кадров в память

Кадр кладётся в память, только если $s^{\text{mask}} > 0.5$, объект виден ($o > 0$) и $s^{\text{kf}} > 0$. Калман включается после 15 стабильных кадров — до этого его прогноз ещё ненадёжен.

Без дообучения и почти без затрат: LaSOT AUC 68.5 $\rightarrow$ 74.2 (SAM 2.1-L $\rightarrow$ SAMURAI-L).

3. SAM2Long: не решать сразу

Ding et al., 2025



Идея как в beam search у языковых моделей: держим $P = 3$ гипотезы-«пути» сразу. У каждого пути своя память. На кадре каждый путь даёт 3 маски $\Rightarrow$ 9 кандидатов, оставляем 3 лучших по *накопленному* счёту. Ошибка на одном кадре не губит трек: правильный путь может выжить и победить позже.

3. SAM2Long: формулы

Накопленный счёт пути

$$S_{p,k}[t] = S_p[t-1] + \log\left(\hat{u}_t^{p,k} + \epsilon\right)$$

$S_p[t-1]$ — счёт пути p до кадра t; $\hat{u}_t^{p,k}$ — предсказанный IoU k-й маски этого пути; ϵ — малая константа против $\log 0$. Сумма логарифмов = логарифм произведения: путь хорош, если хороши *все* его кадры.

Память пути: только надёжные кадры

Идём назад от $t-1$ и берём кадр i в память, только если $\hat{u}_i > 0.3$ и $o_i > 0$, пока не наберём N кадров. Кадр 1 — всегда. Ключи кадров памяти масштабируются весами от 0.95 до 1.05: увереннее объект виден — больше вес.

Если все пути не уверены ($\max_p |o_t^p| < 2$), оставляем кандидатов только с *разными* IoU — чтобы гипотезы не схлопнулись в одну.

3. SAM2Long: что даёт

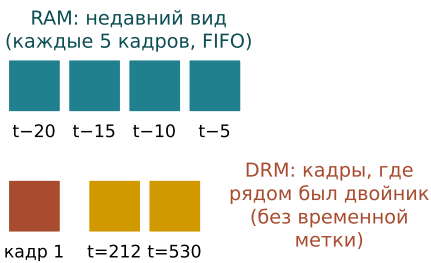
База $\rightarrow$ SAM2Long	SA-V val	SA-V test	LVOS v2 val
SAM 2-L	76.3 $\rightarrow$ 80.8	75.5 $\rightarrow$ 80.8	83.0 $\rightarrow$ 85.2
SAM 2.1-L	78.6 $\rightarrow$ 81.1	79.6 $\rightarrow$ 81.2	84.0 $\rightarrow$ 85.3
SAM 2.1-S	76.9 $\rightarrow$ 79.6	76.9 $\rightarrow$ 80.4	82.1 $\rightarrow$ 84.3

$\mathcal{J}\&\mathcal{F}$, %. Ding et al. SAM2Long, ICCV 2025, arXiv 2410.16268

- В среднем $+3.0$ по 24 сравнениям, максимум $+5.3$. Без дообучения.
- Цена: энкодер считается один раз, а декодер и память — на каждый путь. При $P = 3$: на 18% медленнее, $+8$% FLOPs. $P = 4$ уже не помогает.

Больше всего выигрывают длинные видео (SA-V, LVOS), где одна ошибка у SAM 2 тянется сотни кадров.

3. DAM4SAM: память для двойников Videnovic et al., 2025





- **RAM** (recent appearance memory) — как выглядит объект сейчас: FIFO, но обновляется раз в $\Delta = 5$ кадров и только если маска не пустая.
- **DRM** (distractor-resolving memory) — кадр 1 и кадры-«якоря», где рядом был двойник; хранятся без временной метки, как «вечные» подсказки.

3. DAM4SAM: как заметить двойника

Тест по трём маскам SAM 2

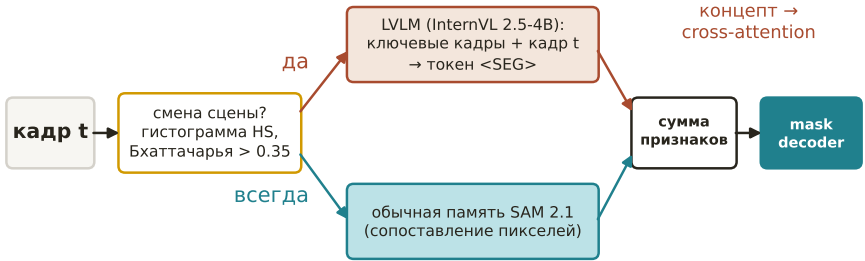
$$\frac{\text{Area}\big(\text{box}(\hat{M}_{k^*})\big)}{\text{Area}\big(\text{box}(\hat{M}_{k^*} \cup \hat{M}_{\text{alt}})\big)} < 0.7 \;\Rightarrow\; \text{рядом двойник}$$

$\hat{M}_{k^*}$ — выбранная маска; $\hat{M}_{\text{alt}}$ — альтернативные маски из той же тройки; $\text{box}(\cdot)$ — описанный бокс; Area — его площадь.

- Альтернативная маска «нашла» что-то похожее *в стороне* $\Rightarrow$ бокс объединения намного больше $\Rightarrow$ это момент, который стоит запомнить.
- Кадр кладётся в DRM, только если трекинг стабилен (предсказанный IoU выше порога) — иначе запомним ошибку.

Модель использует *собственные сомнения* как сигнал (introspection). LaSOT AUC 75.1; второе место VOTS 2025; поверх лёгкого EfficientTAM $+11\%$.

3. SeC: добавить смысл объекта Zhang et al., 2025



Человек после смены плана узнаёт «того же человека в красной куртке», даже если ни один пиксель не похож. SeC (Segment Concept) строит такой *концепт объекта* большой визуально-языковой моделью (LVLM) по нескольким ключевым кадрам и подмешивает его к признакам SAM 2.1.

3. SeC: формулы и расписание LVLM

Детектор смены сцены

h_t — нормированная 2D-гистограмма кадра по тону и насыщенности (HSV). Расстояние Бхаттачарьи:

$$d_B(h_t, h_{t-1}) = \sqrt{1 - \sum_b \sqrt{h_t(b)\, h_{t-1}(b)}}, \qquad d_B > 0.35 \Rightarrow \text{новая сцена}$$

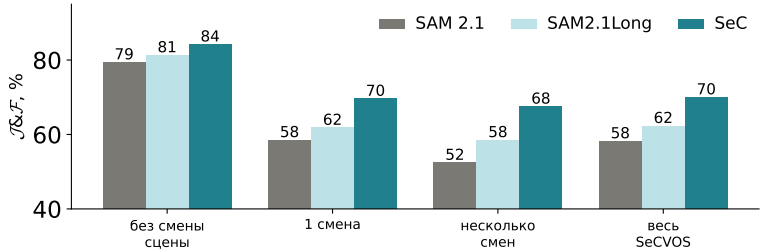
b — ячейка гистограммы. Одинаковые гистограммы дают 0, не пересекающиеся — 1.

Концепт и слияние

LVLM (InternVL 2.5-4B) получает ключевые кадры + кадр t и специальный токен $\langle\text{SEG}\rangle$. Его скрытое состояние $\mathbf{c}$ — концепт. Признаки кадра: $\mathbf{X}' = \mathbf{X}_{\text{mem}} + \text{CrossAttn}(\mathbf{f}_t, \mathbf{c})$.

$\mathbf{X}_{\text{mem}}$ — выход обычной памяти SAM 2.1. LVLM запускается только на смене сцены: 7.4% кадров на SeCVOS и 1% на SA-V — поэтому SeC почти так же быстр, как SAM 2.1.

3. SeC: где помогает концепт



SeCVOS, 160 видео со сменами сцен. Zhang et al. SeC, arXiv 2507.15852

Без смены сцены выигрыш скромный (79.4 $\rightarrow$ 84.2), с несколькими сменами — огромный (52.4 $\rightarrow$ 67.5). Итого $+11.8$ к SAM 2.1. На стандартных наборах: SA-V test 79.6 $\rightarrow$ 81.7, MOSEv2 49.5 $\rightarrow$ 53.8.

3. Сводка улучшений SAM 2

Метод	Что меняет	Цена	Когда брать
SAMURAI	выбор маски + отбор памяти по движению	почти ноль	быстрый объект, двойники
SAM2Long	P гипотез с накопленным счётом	$+18\%$ времени	длинное видео
DAM4SAM	память RAM + DRM, тест на двойника	почти ноль	толпа, похожие объекты
SeC	концепт от LVLM на смене сцены	LVLM на 1–7% кадров	монтаж, смена планов
SAM 2.1	переобучение: 16 кадров, аугментации	новые веса	база для всего

Все четыре метода — про *управление памятью*, а не про новую сеть. В 2026 эта линия продолжается: гибридные памяти (короткая по виду + длинная по двойникам) работают и поверх SAM 3.

4. SAM 3 и SAM 3.1: что вышло Meta, 2025–2026

SAM 3 (ноябрь 2025): сегментация по концепту

Дано: фраза («жёлтый школьный автобус») и/или примеры-боксы.
Найти: маски *всех* таких объектов, в видео — с постоянными ID; если их нет — пустой ответ.

- SAM 2: клик $\rightarrow$ один объект. SAM 3: фраза $\rightarrow$ сколько угодно объектов, включая ноль.
- Устройство: общий энкодер для картинки и текста + детектор по фразе + **трекер с архитектурой SAM 2**. Всё разобранное сегодня внутри SAM 3 остаётся.
- SAM 3.1 (март 2026): много объектов ведутся совместно, $\approx 7\times$ быстрее при 128 объектах.
- Как трекер по маске кадра 1 (MOSEv2): SAM 2.1 — 49.5, SAM 3 — 60.3, SAM 3.1 — 62.3.

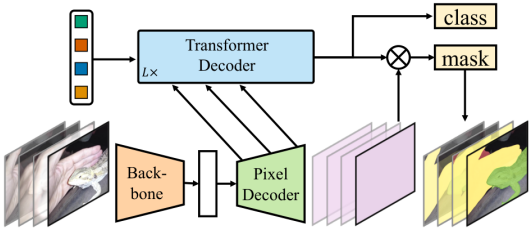
Подробно — в дополнительной части в конце лекции, если останется время.

5. Сегментация по тексту (referring VOS)

детектор + SAM 2	Grounding DINO на кадре 1 → бокс → SAM 2	просто; всё решает кадр 1
адаптер внутри SAM 2	SAMWISE: текст + время в признаках SAM 2	MeViS 37.7 → 48.3
MLLM + SAM 2	Sa2VA, VideoLISA: LLM выдаёт токен [SEG]	сложные фразы
SAM 3	фраза = концепт, все экземпляры	простые именные фразы

- Трудные фразы описывают *движение*: «птица, которая взлетает последней». По одному кадру не решить — нужен весь клип (MeViS).
- SAMWISE (CVPR 2025): MeViS 48.3 против 37.7 у связки Grounding DINO + SAM 2 по первому кадру.
- MLLM + SAM 2 (Sa2VA и др.): языковая модель понимает фразу и выдаёт токен [SEG], из которого декодер SAM 2 строит маску — тот же приём, что в SeC.

5. VIS: все объекты классов сразу



Cheng et al. Mask2Former-Video, arXiv 2112.10764

Запрос = трубка

Mask2Former для клипа: N запросов смотрят на все T кадров сразу; каждый даёт класс и 3D-маску $M(n,t,\mathbf{p}) = \sigma\left(\mathbf{e}_n^\top \mathbf{g}_t(\mathbf{p})\right)$ — объект n во всех кадрах t с одним ID.

$\mathbf{e}_n$ — эмбеддинг n-го запроса; $\mathbf{g}_t(\mathbf{p})$ — признак пикселя $\mathbf{p}$ кадра t. YouTube-VIS 2019: 60.4 AP.

Клип целиком не влезает в память для длинного видео, поэтому на практике — онлайн-варианты (MinVIS, DVIS): запросы сопоставляют от кадра к кадру. Для открытого словаря в 2026 всё чаще просто SAM 3.1 по названию класса.

6. Метрики: $\mathcal{J}$ — площадь, $\mathcal{F}$ — граница

Region similarity $\mathcal{J}$ (IoU)

$$\mathcal{J} = \frac{|M \cap G|}{|M \cup G|}$$

M — предсказанная маска, G — истинная; $|\cdot|$ — число пикселей.

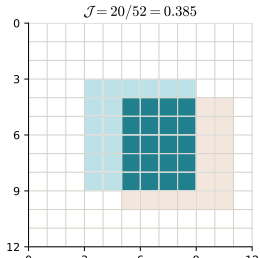
Contour accuracy $\mathcal{F}$

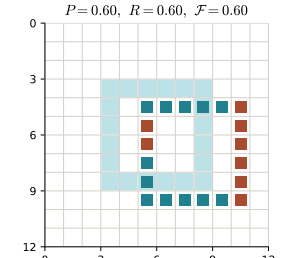
$$P = \frac{|\partial M \cap \mathcal{N}_\tau(\partial G)|}{|\partial M|}, \quad R = \frac{|\partial G \cap \mathcal{N}_\tau(\partial M)|}{|\partial G|}, \quad \mathcal{F} = \frac{2PR}{P+R}$$

$\partial M, \partial G$ — пиксели границы масок; $\mathcal{N}_\tau(\cdot)$ — полоса шириной τ вокруг границы; P — доля границы предсказания рядом с истинной, R — наоборот. В DAVIS $\tau = 0.008 \cdot \sqrt{H^2 + W^2}$ — около 8 пикселей при 480p.

$\mathcal{J}\&\mathcal{F} = \frac{1}{2}(\bar{\mathcal{J}} + \bar{\mathcal{F}})$ — среднее по кадрам и объектам. $\mathcal{J}$ не замечает «лохматой» границы крупного объекта, $\mathcal{F}$ — её главная цель.

6. Пример: квадрат, сдвинутый на (1, 2) пикселя





- Истина G: квадрат 6×6, $|G| = 36$. Предсказание M: тот же квадрат на 1 вниз и 2 вправо, $|M| = 36$.
- Пересечение: $5 \times 4 = 20$; объединение: $36 + 36 - 20 = 52$; $\mathcal{J} = 20/52 = 0.385$.
- Граница квадрата — 20 пикселей. При $\tau = 1$ в полосу вокруг ∂G попали 12 из 20: $P = R = 0.6$, $\mathcal{F} = 0.6$, $\mathcal{J}\&\mathcal{F} = 0.49$.

6. Тонкости метрик

- **Пустая маска на пустом кадре**: если объекта нет и модель ничего не нарисовала, $\mathcal{J} = \mathcal{F} = 1$ по соглашению DAVIS. Нарисовала лишнее — 0.
- **Мелкие объекты**: сдвиг на пиксель у мелкого объекта губит $\mathcal{J}$ сильнее. MOSEv2 делает допуск границы зависимым от площади:

$$\tau = \min\bigl(0.008\,D,\ 0.1\sqrt{A}\bigr) \quad \Rightarrow \quad \dot{\mathcal{F}}$$

  D — диагональ кадра, A — площадь объекта в пикселях.
- **Исчезновение и возвращение**: MOSEv2 отдельно считает $\mathcal{J}\&\dot{\mathcal{F}}_d$ (клипы, где объект исчезает) и $\mathcal{J}\&\dot{\mathcal{F}}_r$ (где возвращается).
- **YouTube-VOS**: $\mathcal{G}$ — среднее $\mathcal{J}$ и $\mathcal{F}$ по известным и новым категориям.
- **VIS**: AP как в COCO, но IoU считается по всей трубке: $\sum_t |\hat{m}_t \cap m_t| / \sum_t |\hat{m}_t \cup m_t|$.

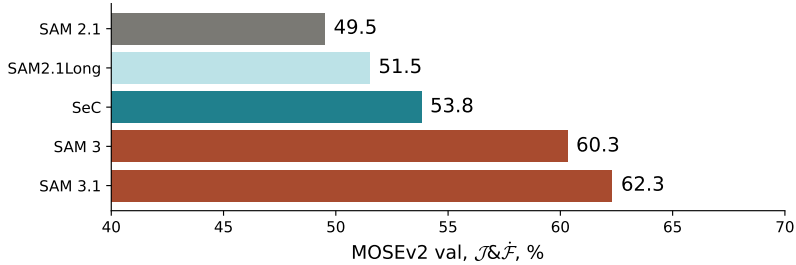
7. Датасеты

Датасет	Видео	Объектов	Масок	Особенность
DAVIS 2017	150	376	10.5K	480p, почти насыщен
YTVOS 2019	4 453	≈7.8K	≈197K	94 категории, метрика $\mathcal{G}$
MOSE (2023)	2 149	5 200	432K	толпы, окклюзии, исчезновения
SA-V (2024)	50 900	642.6K	35.5M	разметка через SAM 2, 47 стран
MOSEv2 (2025)	5 024	10 074	702K	200 категорий, исчезают 61.8%
SeCVOS (2025)	160	—	—	смены сцен, нужен «смысл»

Числа из статей датасетов; YouTube-VOS — приблизительно

Датасеты устаревают: DAVIS решён (≈93), поэтому сравнивать методы 2026 года стоит на MOSEv2, LVOS и SA-V.

7. Состояние в 2026: самый трудный бенчмарк



SAM 2.1, SAM2.1Long, SeC: статья SeC; SAM 3 / 3.1: RELEASE_SAM3p1.md

Улучшения памяти дают $+2 \dots 4$, новое поколение модели и данных — $+11$. На соревновании LSVOS 2026 (ECCV) в треке MOSEv2 решения строятся поверх SAM 3: третье место — SAM3Dual с двумя ветками памяти без дообучения, 64.37. До DAVIS-уровня (90+) ещё очень далеко.

7. Как выбрать инструмент



- Сначала — SAM 2.1 или SAM 3.1 с клика; посмотрите, где ломается.
- Ломается на двойниках или после окклюзий ⇒ SAMURAI, DAM4SAM, SAM2Long — меняется только код памяти, веса те же.
- Монтаж и смена планов ⇒ SeC. Много объектов ⇒ SAM 3.1 Multiplex.

Главные выводы

1. Главная идея VOS-трекеров — память + сопоставление (attention): кадр сразу связан со всеми запомненными, включая кадр 1 с истиной.
2. SAM 2 = SAM для кадра + memory attention + memory bank (6 последних + кадры с промптами + object pointers) + голова окклюзии. Прорыв — в данных SA-V и data engine.
3. Слабое место SAM 2 — *правила памяти*: FIFO, жадный выбор маски, нет смысла объекта.
4. Современные улучшения меняют правила без дообучения: движение (SAMURAI), несколько гипотез (SAM2Long), память для двойников (DAM4SAM), концепт от LVLM (SeC).
5. Вышли SAM 3 и SAM 3.1: фраза $\rightarrow$ все экземпляры, трекер унаследован от SAM 2; как трекер заметно сильнее SAM 2.1.
6. Метрики: $\mathcal{J}$ — площадь, $\mathcal{F}$ — граница; на MOSEv2 даже лучшие модели около 60–64 — задача далека от решения.

Дополнительно: SAM 3 подробно

Разбираем, если осталось время

- постановка PCS и архитектура SAM 3;
- presence token: «есть ли» отдельно от «где»;
- SAM 3 на видео: детектор + трекер и две страховки;
- метрика cgF1, SAM 3.1 Multiplex, результаты как трекера.

Д. SAM 3: «все такие объекты» Carion et al., 2025

Promptable Concept Segmentation (PCS)

Дано: картинка или видео и *концепт* — короткая именная фраза («жёлтый школьный автобус») и/или примеры-боксы.
Найти: маски *всех* экземпляров концепта, в видео — с постоянными ID. Если концепта нет — пустой ответ.

- SAM 2: один клик → один объект. SAM 3: одна фраза → сколько угодно объектов, включая ноль.
- Модель: 848M параметров; один энкодер Perception Encoder (PE) для картинки и текста; детектор + трекер.
- Данные SA-Co: 4M уникальных фраз и 52M масок с людьми, плюс 1.4B синтетических масок; видео — 52.5K роликов, 467K masklets.

Старые трюки «детектор по тексту + SAM 2» (Grounded-SAM) SAM 3 заменяет одной моделью, обученной с трудными отрицательными примерами.

Д. SAM 3: архитектура

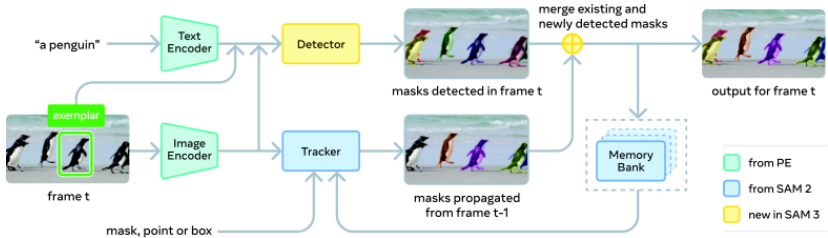
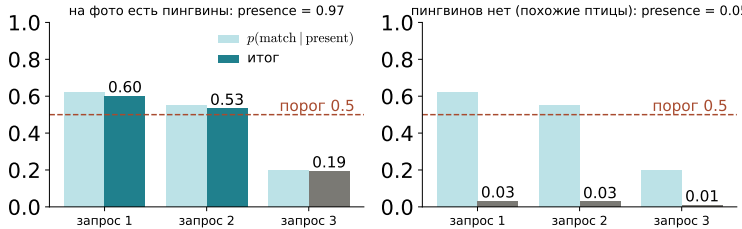


Рисунок: Carion et al. SAM 3, arXiv 2511.16719. Зелёное — из PE, синее — из SAM 2, жёлтое — новое

- **Fusion encoder**: признаки кадра cross-attention’ом смотрят на токены фразы и примеров.
- **DETR-декодер**: обучаемые запросы-объекты; каждый даёт бокс, маску и «подходит ли под фразу».
- **Трекер** — архитектура SAM 2 (memory attention, memory bank, mask decoder) на том же энкодере.

Д. Presence token: отделить «есть ли» от «где»

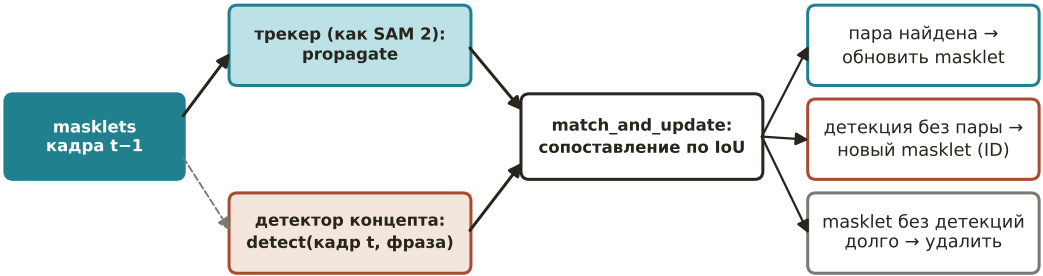


Счёт запроса

$\text{score}_i = p(\text{фраза есть на кадре}) \cdot p(q_i \text{ подходит} \mid \text{фраза есть})$. Первый множитель даёт один глобальный presence token на весь кадр, второй — каждый запрос q_i. Детекция принимается при $\text{score}_i > 0.5$.

Без presence каждый запрос сам решает и «есть ли пингвины», и «я ли пингвин» — и на похожих птицах даёт ложные маски. С ним справа все счета падают ниже порога.

Д. SAM 3 на видео: детектор + трекер



$$\hat{\mathcal{M}}_t = \text{propagate}(\mathcal{M}_{t-1}), \quad \mathcal{O}_t = \text{detect}(I_t, P), \quad \mathcal{M}_t = \text{match_and_update}(\hat{\mathcal{M}}_t, \mathcal{O}_t)$$

$\mathcal{M}_{t-1}$ — masklets прошлого кадра; $\hat{\mathcal{M}}_t$ — они же, протащенные трекером; P — промпт (фраза); $\mathcal{O}_t$ — детекции кадра t. Сопоставление — по IoU масок, как ассоциация в лекции 5.

Д. SAM 3 на видео: две страховки

Masklet detection score

Для каждого masklet считаем, в скольких из последних кадров он совпал с какой-нибудь детекцией. Счёт ниже порога $\Rightarrow$ masklet удаляется: трекер «ведёт» то, чего детектор не видит, — скорее всего, ошибка.

Периодическое перепромптирование

Время от времени уверенные маски детектора *заменяют* маски трекера и записываются в память как кадры с промптом. Память получает свежие надёжные примеры — лекарство от окклюзий и двойников.

Связь с лекцией 5: это tracking-by-detection, но и «детектор», и «трекер» — сегментационные, и у трекера есть собственная память о виде объекта. Как и у SAMURAI, лучшие решения берут сильные стороны обоих подходов.

Д. Метрика SAM 3: cgF1

Две части ошибки

pmF_1 — F1 сопоставления масок на кадрах, где концепт *есть* («нашли ли все экземпляры и точно ли»).
IL_MCC — коэффициент корреляции Мэтьюса для ответа «есть / нет» на уровне кадра:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \in [-1, 1]$$

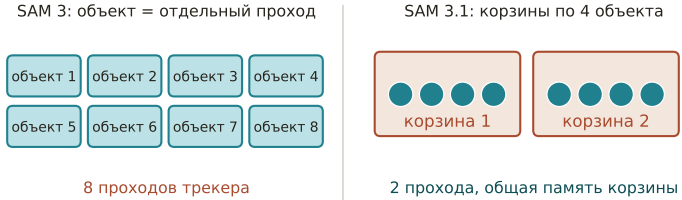
TP — концепт есть и модель сказала «есть»; *TN* — нет и «нет»; *FP* — нет, а сказала «есть»; *FN* — есть, а сказала «нет».

$$\text{cgF}_1 = 100 \cdot \text{pmF}_1 \cdot \text{IL_MCC}$$

Модель, которая рисует маски на каждом кадре «на всякий случай», получит низкий MCC и низкий cgF1. SA-Co/Gold: SAM 3 — 54.1, люди — 72.8.

Д. SAM 3.1: Object Multiplex

Meta, март 2026



- В SAM 3 каждый объект — отдельный проход трекера: время растёт линейно с числом объектов.
- Multiplex группирует объекты в *корзины фиксированной ёмкости* и ведёт объекты корзины совместно, с общей памятью. Плюс меньше синхронизаций CPU–GPU и пакетная постобработка.
- Итог: $\approx 7\times$ быстрее при 128 объектах на одной H100.

Почему это важно: PCS по фразе «человек» на стадионе — это сотни объектов. Без Multiplex SAM 3 в таких сценах непрактичен.

Д. SAM 3 и SAM 3.1 как трекеры (VOS)

Модель	MOSE	DAVIS	LVOS 2	SA-V val	SA-V test	YTVOS	MOSEv2
SAM 2.1-L	74.6	—	80.6	—	79.5	—	49.5
SAM 3	78.4	92.2	88.5	83.5	84.4	89.7	60.3
SAM 3.1	79.6	92.7	89.2	83.8	85.1	89.3	62.3

$\mathcal{J}\&\mathcal{F}$ (MOSEv2 — $\mathcal{J}\&\dot{\mathcal{F}}$), %. SAM 3 / 3.1: RELEASE_SAM3p1.md (facebookresearch/sam3); SAM 2.1: README sam2 и статья SeC (MOSEv2)

Трекер SAM 3 — та же архитектура SAM 2, но обучен на большем и более трудном наборе данных и с более сильным энкодером PE. Как обычный VOS-трекер по маске первого кадра он заметно сильнее SAM 2.1: $+10.8$ на MOSEv2.

Литература (1/2)

- **Ravi et al.** — SAM 2: Segment Anything in Images and Videos, ICLR 2025. arxiv.org/abs/2408.00714; SAM 2.1: github.com/facebookresearch/sam2
- **Carion et al.** — SAM 3: Segment Anything with Concepts, 2025. arxiv.org/abs/2511.16719; SAM 3.1: github.com/facebookresearch/sam3 (RELEASE_SAM3p1.md)
- **Yang et al.** — SAMURAI, 2024. arxiv.org/abs/2411.11922
- **Ding et al.** — SAM2Long, ICCV 2025. arxiv.org/abs/2410.16268
- **Videnovic et al.** — A Distractor-Aware Memory for Visual Object Tracking with SAM2 (DAM4SAM), CVPR 2025. github.com/jovanavidenovic/DAM4SAM
- **Zhang et al.** — SeC: Advancing Complex VOS via Progressive Concept Construction, 2025. arxiv.org/abs/2507.15852
- **Ding et al.** — MOSEv2, 2025. arxiv.org/abs/2508.05630; LSVOS 2026: lsvos.github.io

Литература (2/2)

- **Cheng, Tai, Tang** — STCN, NeurIPS 2021. arxiv.org/abs/2106.05210
- **Cheng, Schwing** — XMem, ECCV 2022. arxiv.org/abs/2207.07115
- **Cheng et al.** — Putting the Object Back into VOS (Cutie), CVPR 2024. arxiv.org/abs/2310.12982
- **Cheng et al.** — Mask2Former for Video Instance Segmentation, 2021. arxiv.org/abs/2112.10764
- **Cuttano et al.** — SAMWISE, CVPR 2025. arxiv.org/abs/2411.17646
- **Pont-Tuset et al.** — DAVIS 2017. arxiv.org/abs/1704.00675; **Xu et al.** — YouTube-VOS. arxiv.org/abs/1809.03327; **Ding et al.** — MOSE. arxiv.org/abs/2302.01872
- **Su et al.** — RoFormer (RoPE), 2021. arxiv.org/abs/2104.09864